

Getting Started with R for Survey Data

George M. Harrison University of Hawai'i at Mānoa, College of Education

January, 2021

Contents

1	What is R	2
2	Download R, then RStudio	2
2.1	How and from where	2
2.2	Look at RStudio's environment	2
3	Creating and re-opening an R script file	4
3.1	Create an R script file in your folder, then start RStudio by double-clicking that file	4
3.2	If you need to set the working directory	4
4	Saving and closing an R session	6
5	Importing data from a CSV file	7
6	Examining the data	9
6.1	Making heads and tails of our data	10
6.2	Point-and-click in RStudio	10
6.3	Viewing the structure of the data	11
6.4	Extracting the names of the columns	12
6.5	Subsetting data using \$ and indexing	12
6.6	Creating objects with the combine function	13
6.7	Putting the functions together	14
7	Describing the data	15
7.1	The summary function	15
7.2	Specific functions for summarizing data	16
8	Creating new variables	17
8.1	Creating scores from items	17
8.2	Appending variables to our data frame	18
9	Creating a simple box plot	19
10	Getting help and troubleshooting	19
10.1	R's help files	19
10.2	Online help	21
10.3	Why won't my code run?	21
10.4	Have patience and keep learning	21
11	List of functions and other code in this tutorial	24

1 What is R

R [R Core Team, 2020] is an open-source language and environment that is primarily used for conducting statistical analyses and managing data. RStudio [RStudio Team, 2020] is an open-source graphical environment for running R, viewing output, and doing other things (like creating this document).

The R community is huge and people develop R packages that we can download (through R) and use for specific types of analyses. In later sessions, we will use some packages, such as the survey package [Lumley, 2020], the srvyr package [Freedman Ellis and Schneider, 2020], and the psych package [Revelle, 2020].

2 Download R, then RStudio

2.1 How and from where

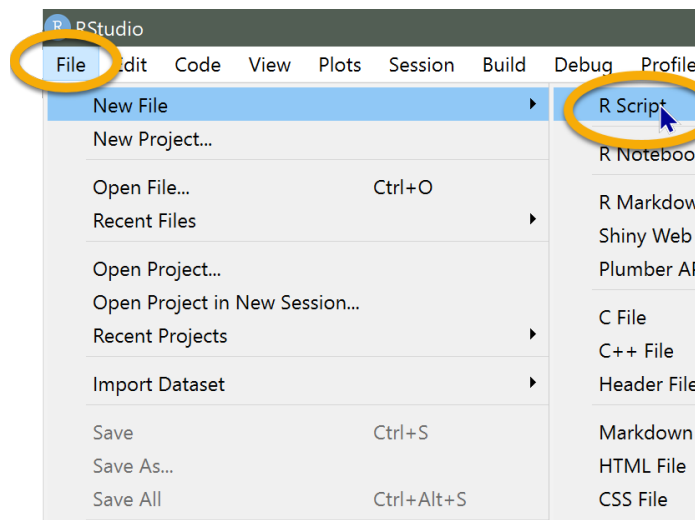
You can view videos on how to download R and RStudio here: <http://go.hawaii.edu/wKf>. On that web page, go to the Set Up panel on the left side of the web page to see the videos. The URLs that the video mentions are here: <https://cloud.r-project.org/> for downloading R and here, <https://www.rstudio.com/products/rstudio/download/> for downloading RStudio.

When you download RStudio, the free open-source license version is all you need. This is labeled as “RStudio Desktop” and is for laptops and desktops.

2.2 Look at RStudio’s environment

Once you have downloaded R and RStudio, you can work in RStudio and forget about the actual R application (until you update it in a few months).

For now, to see what it looks like, open RStudio in the same way you’d open other new programs on your machine (such as through the Start menu on a PC).¹ You’ll see four panes.



If you don’t see the script editor pane, you can create a new script.

You can rearrange the panes, change the size of each, and click on the tabs within each pane to view the contents.

The icon with four boxes within it is for rearranging panes. Sometimes, we might want to have the console (the output) on the right:

¹In your usual workflow, you can open RStudio by clicking on an R file in a local directory on your machine, such as from within a folder you have created for a lesson in our class. We’ll do this next.

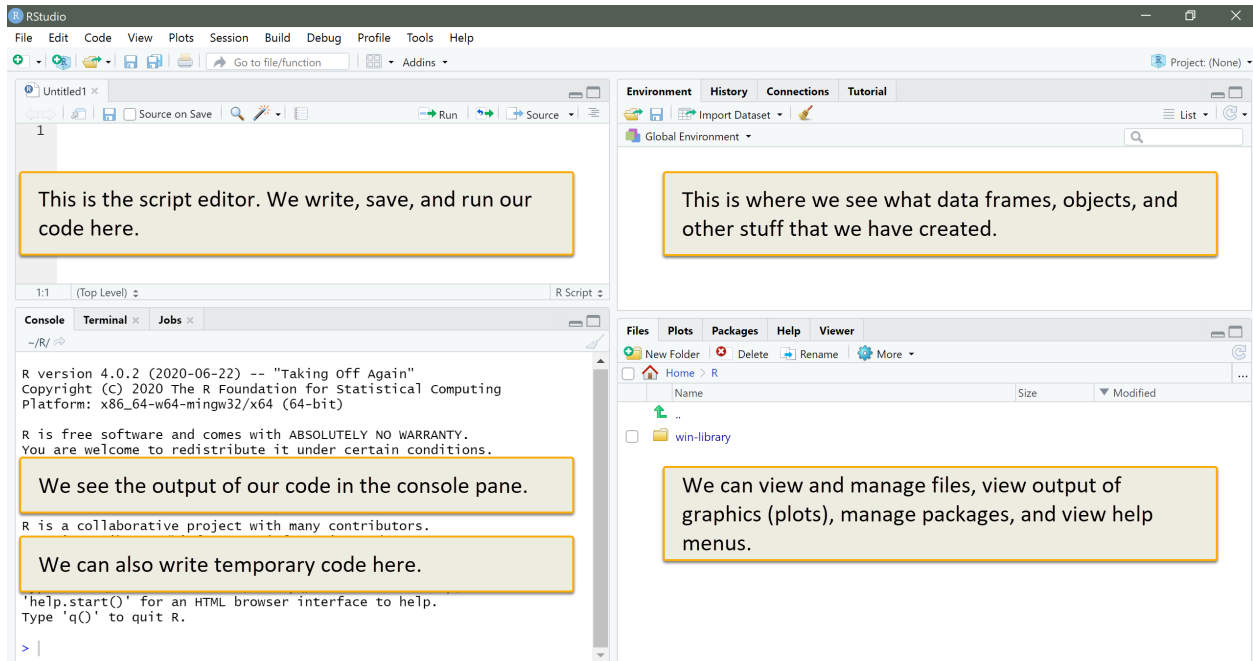


Figure 1: The four panes of the RStudio environment are the script editor, the environment et al., the console., et al., and the files, et al.

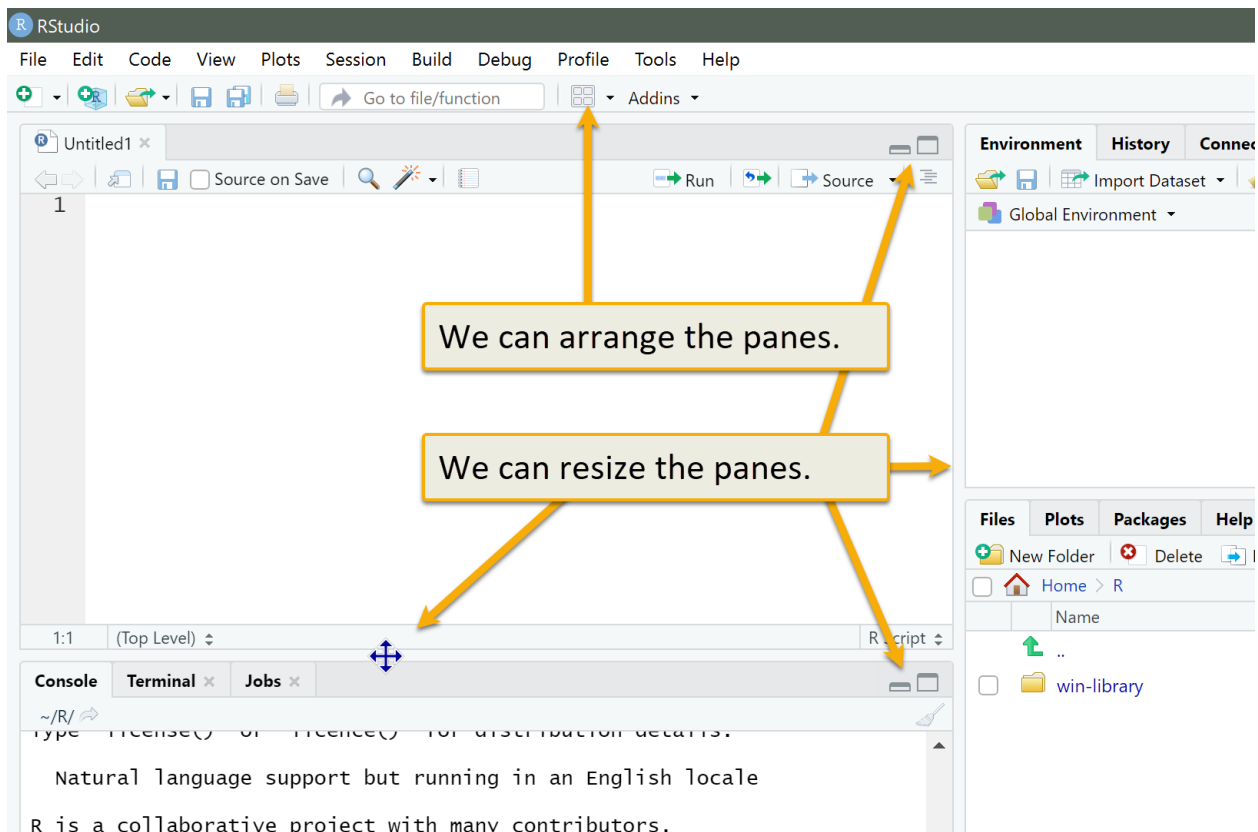


Figure 2: Arranging panes by dragging borders to resize panes and clicking the minimize and maximize icons.

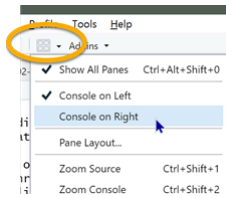


Figure 3: Click the grid icon to select Console on Left or Console on Right.

3 Creating and re-opening an R script file

3.1 Create an R script file in your folder, then start RStudio by double-clicking that file

We typically write code in a script file, run it to be sure our coding is correct, then save the file to a location on our local machine so we can continue working on it another day. For a six-minute introductory video on this process, go to <https://youtu.be/C8sP6rfrMAs>.

There are at least two ways to create a new R script file:

1. Open RStudio to get a blank source script page; then, save the file (using the **File** menu), giving it a name with the extension `.R` and selecting a directory on your computer where you want it to be saved (such as a folder for this class). After doing this, close RStudio. Then, navigate to the folder in which you saved your R script file and open it again by double-clicking on the file. You might need to right click on the file and use **Open With** to direct your computer to open the R file with RStudio.

From there, you should be able to also set your computer to always use RStudio for files with `.R` extensions. By closing RStudio, and then navigating to your folder to open the R file directly, our R session will automatically set the working directory to be that folder location. In other words, this is the lazy way (which I fully appreciate) because we can avoid having to manually set the working directory.



A working directory is the folder location where R looks for files, such as data files, and saves outputted files, such as plots that we output or data that we save. **Advice for lazy folks like me:** Create a folder for each separate analysis. In that folder, place the R script file and the data file (CSV file, which we'll describe in Part 5). This will simplify the process when you tell R which data file to use (because otherwise we have to specify a working directory).

2. Go to your directory where you want to keep the file and create a new document with the `.R` extension.² You might name your file something like `MyAnalysis.R`.³ Open the file. As mentioned above, you might need to use **Open With** and set your computer to always open `.R` files with RStudio.

You can, of course, also place an existing script file in a folder and start a new session of RStudio by clicking on that file. For instance, if you acquired some data and an R script file (from an online source, your instructor, or a colleague), you can place the files into your folder and open the script file from there. This will create the working directory in the right place (as long as RStudio was not already open).

3.2 If you need to set the working directory

Note that when you start an RStudio session from a folder location that does **not** include your data, such as if you access a shortcut on your desktop, you will need create a working directory by entering something like this in your script: `setwd("C:/Users/Yourname/Desktop/Yourfolder")`.⁴

An alternative to using the `setwd()` function is to use RStudio's GUI. So, if you have your data and your R script in a folder, but you're unable to read the data (perhaps because you opened RStudio from your applications menu instead of clicking on the file in your folder), you can do this: Go to the pane that shows your files, plots, packages, and so forth. Open the files tab and navigate to the folder where your data reside by clicking on the ellipsis button `...` on the right. After that, go to the **More** menu and select **Set As Working Directory**.

Now our files tab shows the folder we want (in this example, it's a folder on the desktop).

²On a PC, you can right click in a folder and create a new file, then change the extension from `.txt` to `.R`.

³You might need to tell your computer to display the file extensions of files, such as `.docx` for Word docs and `.R` for R script

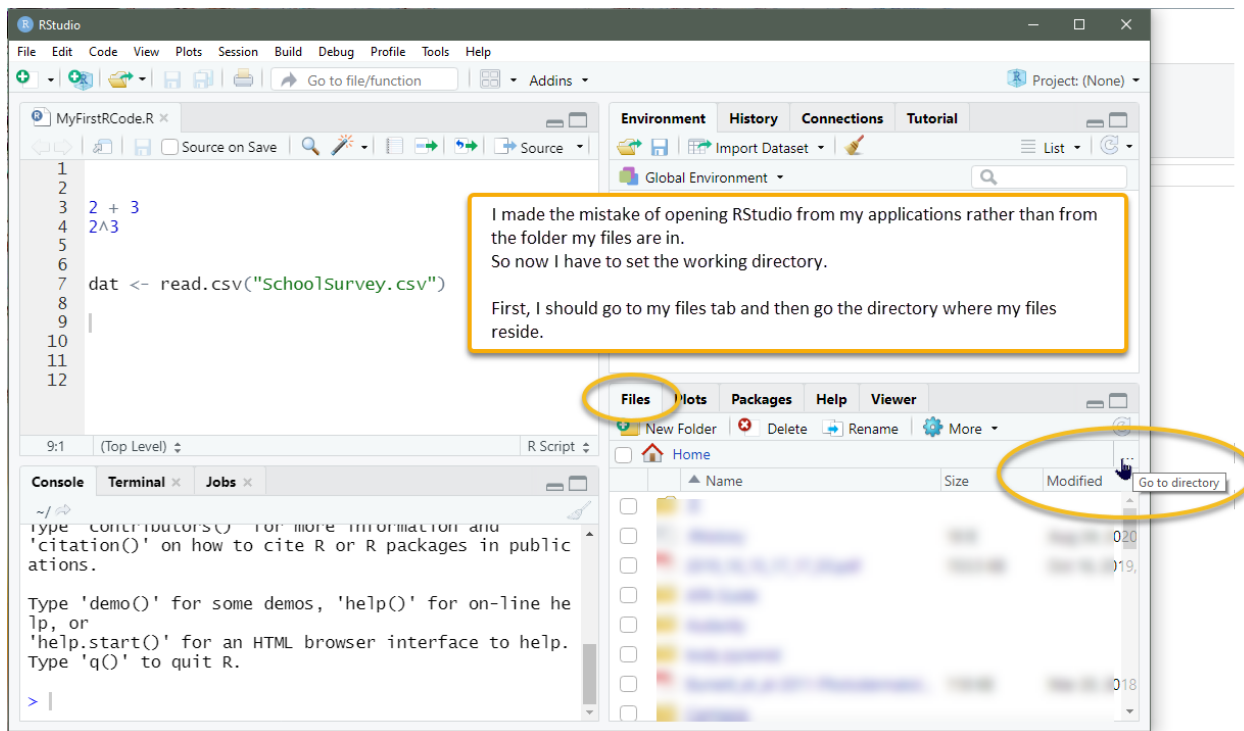


Figure 4: Click the small icon on the top right of the files pane to navigate to the files directory (such as a folder you created for your class or specific analysis).

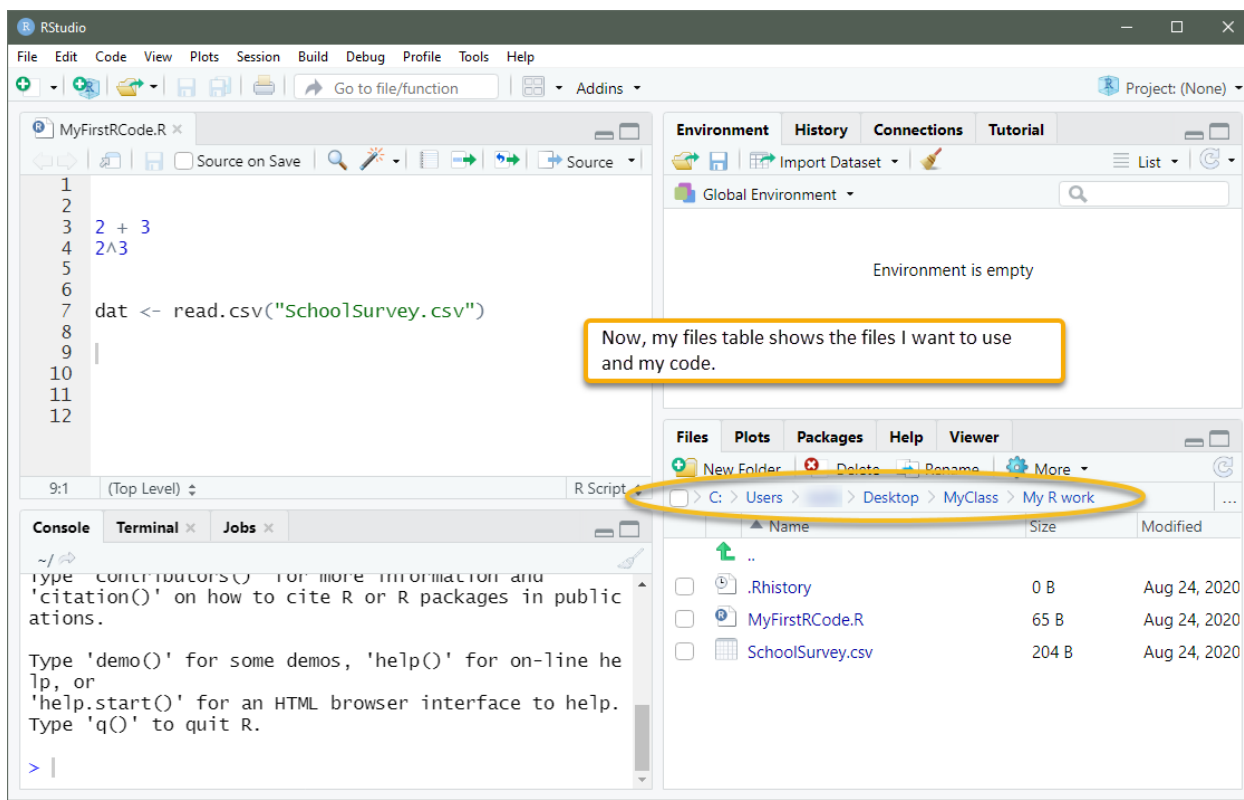


Figure 5: Now our files tab shows the folder we want (in this example, it's a folder on the desktop)

Here's the **More** directory, where **Set As Working Directory** can be selected.

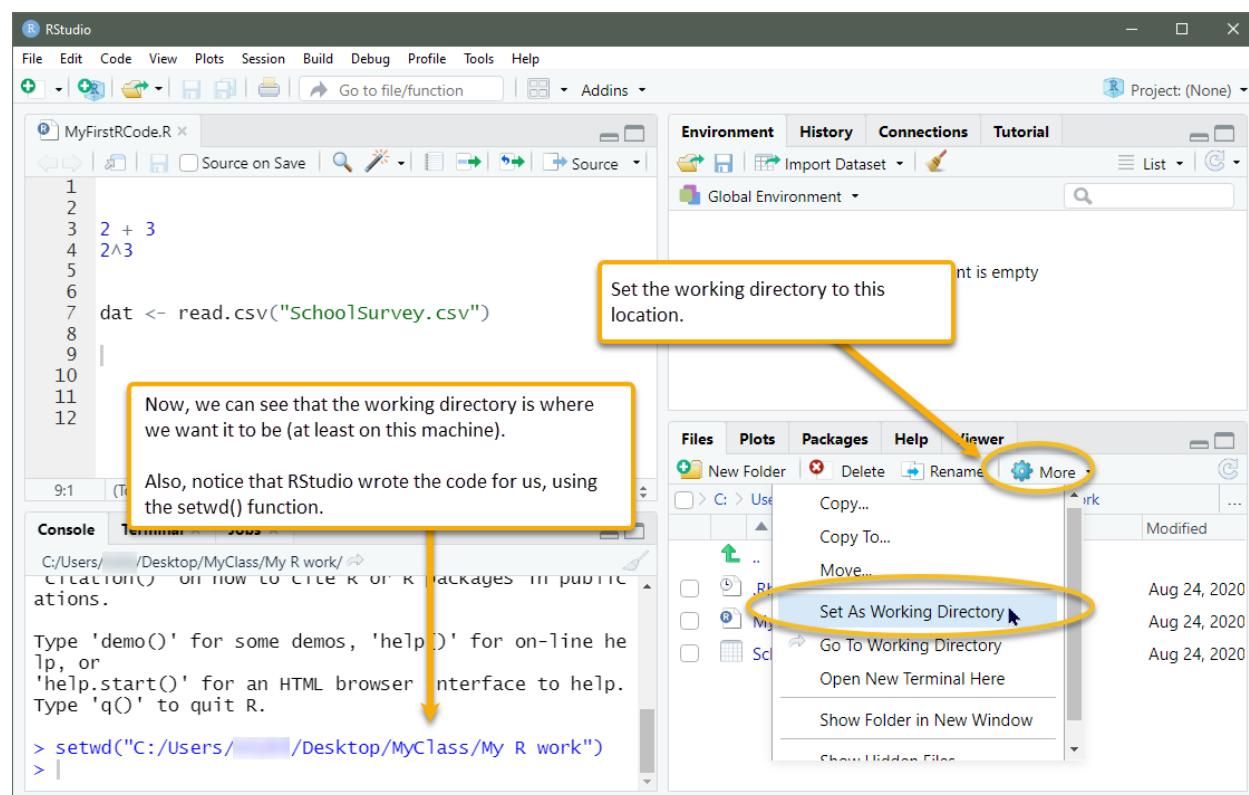


Figure 6: In the Files pane, select the More icon and then select Set as working directory.

Rather than dealing with all this working directory business, it is easier to place all of our files in a single folder and start an RStudio session by clicking on the `.R` file.⁵

4 Saving and closing an R session

Notice that after you type any script in your source pane, the file name in the tab will be red and have an asterisk at the end. This means your edits are unsaved. You can save your script at any time (use **Save** in the **File** menu or the save shortcut, `Ctrl+S` or `Cmd+S` depending on your OS).

After you've saved your work, close RStudio. You might be prompted whether you'd like to also save the workspace image.⁶ If you're like me, you'll say "Wait a minute. I thought I just saved it. What's this?" Basically, we have the choice of **also** saving the output we have generated so far. My advice is to **not** save the workspace image unless (a) you're working on something that takes a lot of processing time (e.g., with very large data sets or complex analyses) or (b) you have neared the completion of your project and are making minor tweaks.

The reason we don't want to save the image is that it is best to start each session anew, without any R objects (assuming you also keep your data file in that same directory).⁷ Then you can run your code and be sure that the mistakes you made in the last session with this file are not included in your current session. If you want to be sure your session is clear, you can go to the **Environment** pane of RStudio and click on the broom icon to sweep your environment clean.⁸

⁵Another option is to use RStudio projects.

⁶The default in RStudio at the time of this writing seems to suppress this behavior.

⁷R objects are described next.

⁸You can also clear the console using the broom in that console.

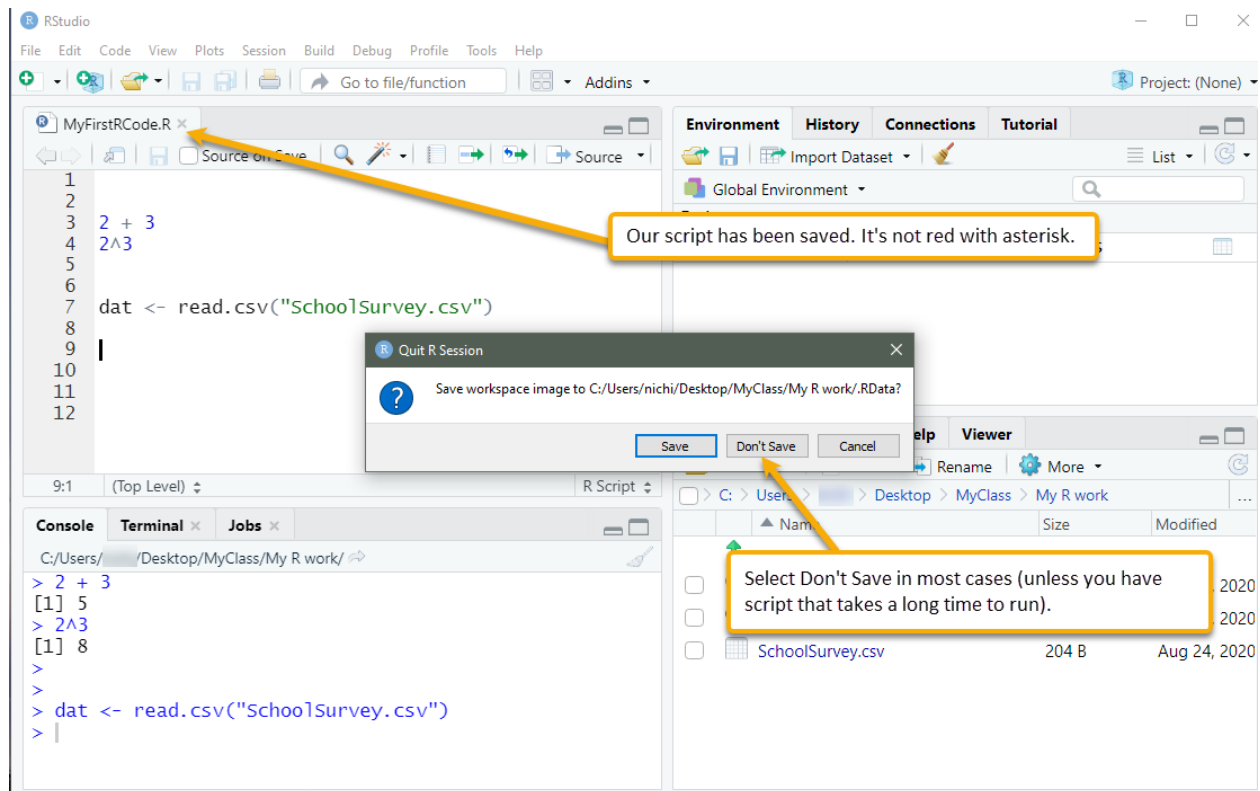


Figure 7: Select “don’t save” if the “Save workspace image to...” prompt appears.

👉 If you accidentally did save an image from a previous R session, there will be stuff in the environment pane. If this stuff contains mistakes, they’re still there even though you restarted R. You should clear your environment and then save the session in its clean state. Then, next time you open it, the environment will be empty.

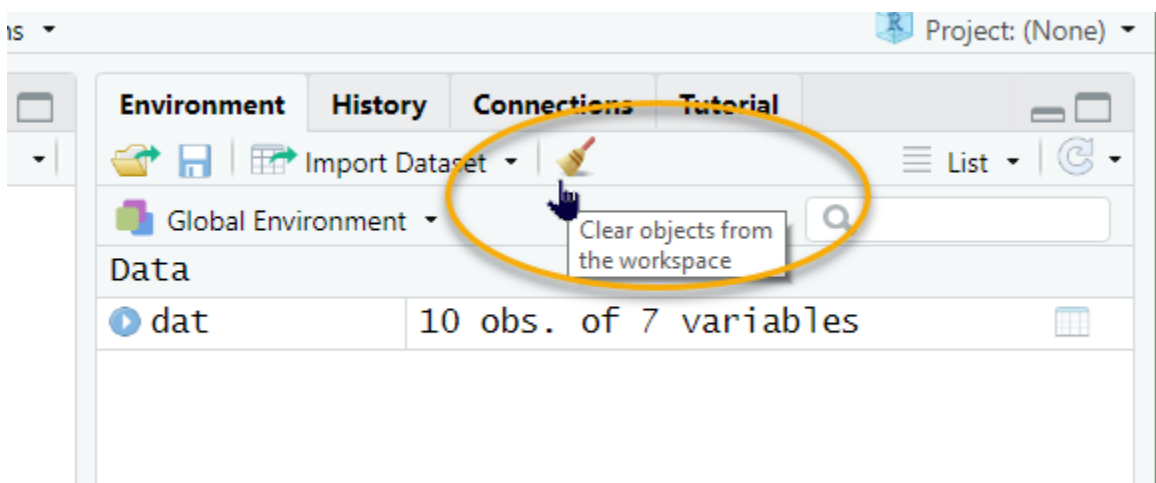


Figure 8: Clicking the broom in the environment pane clears the objects from the workspace.

If you want to stop this prompt from happening in the future, you can change your global settings. Go to Tools and Global Options and change the setting from “Ask” to “Never”:

5 Importing data from a CSV file

Get the data from Laulima and save it to your folder. It is also available here. Our data set contains

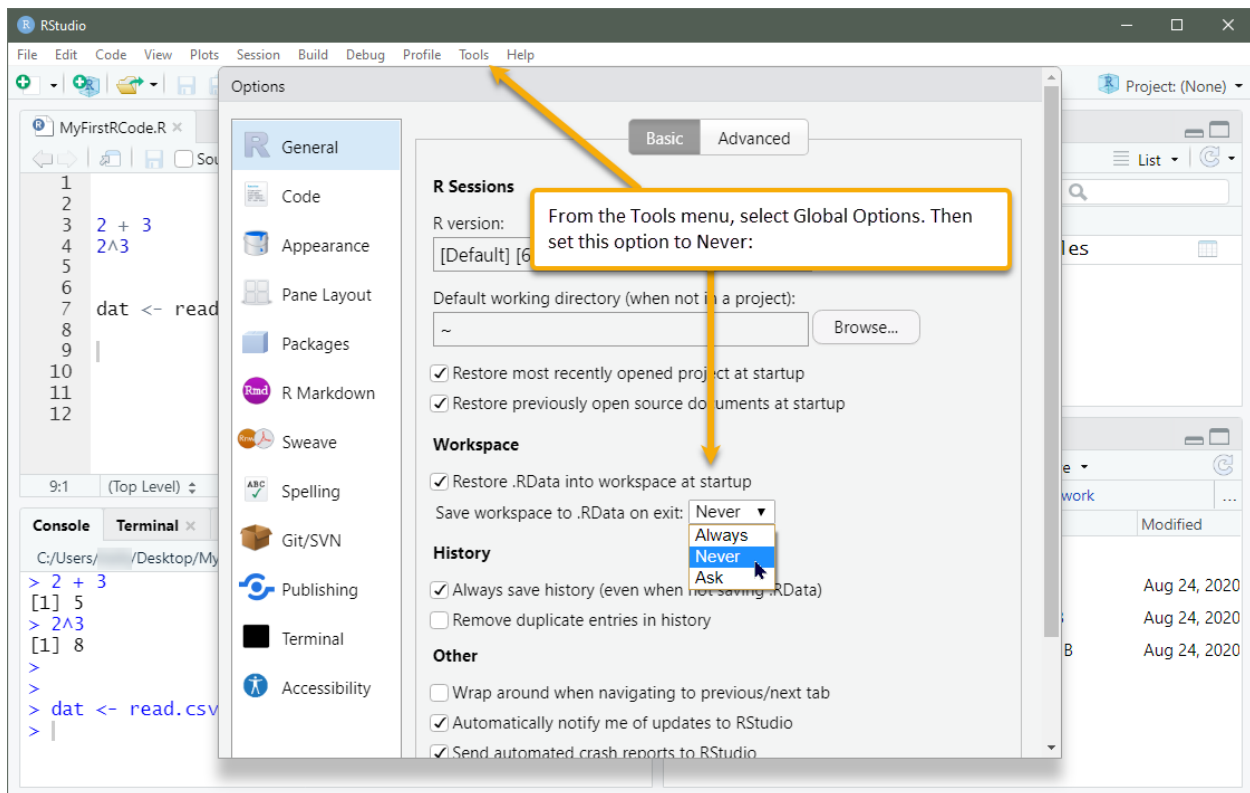


Figure 9: Click Tools, then Global Options, then set the option under Workspace that says Restore .RData into Worspace at startup to “never”.

Let's read the data set into R and save it to an object, which we'll arbitrarily call *dat*. The `read.csv()` function does just that—it reads CSV files.¹⁰ The CSV file needs to be in the same folder as your .R file (or you have set the working directory to that folder, or you have used the directory to the file within quotes). Also, be sure to use quotes around the name. Finally, R is **case sensitive**, so if you get an error, check the spelling. Type the following code in your source pane in RStudio, place your cursor anywhere on the line, and click the Run button.

```
dat <- read.csv("SchoolSurvey.csv")
```



The less-than symbol < and the dash symbol - together as <- are the 'assign to' operation in R. We have instructed R to read the CSV file that is in our current directory (or folder) and assign that to an object labeled 'dat' in our R session.

Notice that nothing seems to show up. We don't see our data. Actually, we did create the object `dat`. It's just that we don't see it unless we ask for it. So, let's now examine the object we just created.

If you were unable to download the data, you can also try this code to directly download the file from the web site:

```
dat <- read.csv("http://www2.hawaii.edu/~georgeha/Handouts/SchoolSurvey.csv")
```

6 Examining the data

Let's make sure our data set was actually imported and that it was formatted in the way we expect. If we have a small data frame, as we do here, we can simply type a new line with our object, `dat`, select the object with our cursor, and run it to view the output in the console.¹¹

```
dat
```



In R, a data set is called a **data frame**. There are other types of objects in R; for example, a **vector** is a single column (or row) of data.

If all went well, we should see this as our output:

##	PID	Lik1	Lik2	Lik3	Lik4	Lik5	Teacher
## 1	1	5	5	4	3	2	No
## 2	2	2	2	3	1	2	Yes
## 3	3	4	4	3	3	2	No
## 4	4	2	2	2	1	2	Yes
## 5	5	5	5	3	5	4	No
## 6	6	1	1	2	2	3	Yes
## 7	7	1	2	3	1	1	Yes
## 8	8	4	1	3	4	5	No
## 9	9	5	3	4	4	3	No
## 10	10	2	2	3	3	4	Yes

¹⁰CSV files are comma-separated-value files, which we might have saved from a worksheet in a spreadsheet program such as Excel.

¹¹When you run this code, you'll see the output in your console. In this handout (but not in your actual R session), it is displayed with double hash marks preceding each line.

6.1 Making heads and tails of our data

Let's also look at the first six lines of data by performing the `head()` function on the data-frame object we just created. This is useful if we have a large data set and we simply want to see whether the variable names and first several observations look the way we expect them to.

```
head(dat)
```

```
##   PID Lik1 Lik2 Lik3 Lik4 Lik5 Teacher
## 1   1    5    5    4    3    2      No
## 2   2    2    2    3    1    2      Yes
## 3   3    4    4    3    3    2      No
## 4   4    2    2    2    1    2      Yes
## 5   5    5    5    3    5    4      No
## 6   6    1    1    2    2    3      Yes
```

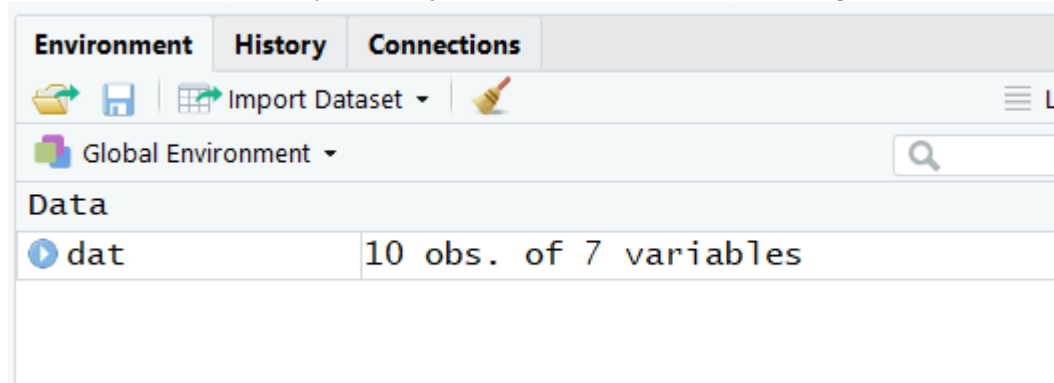
We can also examine the last six observations of our data frame using the `tail()` function.

```
tail(dat)
```

```
##   PID Lik1 Lik2 Lik3 Lik4 Lik5 Teacher
## 5    5    5    5    3    5    4      No
## 6    6    1    1    2    2    3      Yes
## 7    7    1    2    3    1    1      Yes
## 8    8    4    1    3    4    5      No
## 9    9    5    3    4    4    3      No
## 10  10    2    2    3    3    4      Yes
```

6.2 Point-and-click in RStudio

A point-and-click way we can examine our data frame, and any other objects in our current session, is through



the environment pane in RStudio.

If we click on an object itself, such as on the `dat` here, RStudio will open a new tab in the script-editor pane and reveal the object.¹²

¹²You can also use the `View()` function to open a data frame in a new tab and view it. Try the code `View(dat)`.

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function

R Handout Getting Started with R.Rmd x dat x

Filter

	ID	Lik1	Lik2	Lik3	Lik4	Lik5	Teacher
1	1	5	5	4	3	2	No
2	2	2	2	3	1	2	Yes
3	3	4	4	3	3	2	No
4	4	2	2	2	1	2	Yes
5	5	5	5	3	5	4	No
6	6	1	1	2	2	3	Yes
7	7	1	2	3	1	1	Yes
8	8	4	1	3	4	5	No
9	9	5	3	4	4	3	No
10	10	2	2	3	3	4	Yes

With large data sets, this can take a while to load so this is not a typical part of the work flow, but for people who are used to SPSS, this brings some familiarity. Close that tab when you are done, or simply click back on the tab that contains the script you have written so far.

6.3 Viewing the structure of the data

Environment	History	Connections
Import Dataset		
Global Environment		
Data		
dat		10 obs. of 7
ID	: int	1 2 3 4 5 6 7 8 9 10
Lik1	: int	5 2 4 2 5 1 1 4 5
Lik2	: int	5 2 4 2 5 1 2 1 3
Lik3	: int	4 3 3 2 3 2 3 3 4
Lik4	: int	3 1 3 1 5 2 1 4 4
Lik5	: int	2 2 2 2 4 3 1 5 3
Teacher	: Factor	w/ 2 levels

If we click on the arrow icon in the environment pane, we can see the details.

Alternatively, we can use the structure function, `str()`, to get the same information:

```
str(dat)

## 'data.frame':    10 obs. of  7 variables:
## $ PID      : int  1 2 3 4 5 6 7 8 9 10
## $ Lik1     : int  5 2 4 2 5 1 1 4 5 2
## $ Lik2     : int  5 2 4 2 5 1 2 1 3 2
## $ Lik3     : int  4 3 3 2 3 2 3 3 4 3
## $ Lik4     : int  3 1 3 1 5 2 1 4 4 3
## $ Lik5     : int  2 2 2 2 4 3 1 5 3 4
## $ Teacher: chr  "No" "Yes" "No" "Yes" ...
```

Notice that our data comprise 10 observations of 7 variables. Also notice the list of variables. Next to each, after the colon, is a code that tells us what the variable's type is. Here, we have several variables listed as integers (`int`), which means they are numeric. We have one variable, `Teacher`, that is listed either as a character (`chr`) variable or a factor type of variable (`Factor w/ 2 levels "No","Yes"`), depending on which version of R you are running.¹³ The character variable is a string. There can be many possible entries. A factor variable has levels, just like factors in ANOVA. Here, we would have two levels. Factor levels are coded under the hood with numbers, which by default match their alphabetical order, so here a “No” is coded as 1 and a “Yes” is coded as 2. The next set of columns are the first several observations of data (here, all 10), so the first person's PID was 1, she or he selected 5 on the first two Likert-type questions, 4 on the third, and 3 and 2 on the next two questions. This person also selected No, coded as 1, indicating that the individual is not a teacher.

 Numeric variables are coded as `int` for integers (i.e., numbers without a decimal, such as whole numbers) or `num` for numeric (i.e., numbers that can have a decimal). Factors are nominal variables with two or more levels. The factor level numbers merely indicate distinction among the levels rather than any ordered value. (However, later we will encounter ordered factors. An “ordered factor” is R terminology for a variable that is on an ordinal level scale of measurement.)

6.4 Extracting the names of the columns

Sometimes, we need to know the names of the variables. For this, we can use the `names()` function.

```
names(dat)

## [1] "PID"      "Lik1"     "Lik2"     "Lik3"     "Lik4"     "Lik5"     "Teacher"
```

For extra fun, we can ask that the names of the output be in a format that is amenable to copying and pasting (we'll see the value of this in a minute). The `dput()` function does this.¹⁴

```
dput(names(dat))

## c("PID", "Lik1", "Lik2", "Lik3", "Lik4", "Lik5", "Teacher")
```

Notice that the output includes commas between the variables. This will be convenient in a minute.

6.5 Subsetting data using `$` and indexing

Maybe we wish to examine a single variable in our data. Let's look at the first Likert-type item.

¹³The `read.csv()` function's defaults changed in one of the R versions so that character type of column data are read in as character instead of factors.

¹⁴Notice that we can place functions inside of functions. The order of operations is the same as in mathematics, where the inside parentheses, `names(dat)`, are evaluated before the outside parentheses, `dput(...)`.

```
dat$Lik1
```

```
## [1] 5 2 4 2 5 1 1 4 5 2
```



The dollar sign is used to select a particular column (i.e., variable) from the data. All rows of this column of data are included. Because we've selected a single column, the result is a *vector* of numbers.

The dollar sign is one way to select a particular column in our data frame. Another way is to use indexing, with brackets. Within the brackets are two indices. The first is the row index and the second is the column index. If an index entry is blank, it instructs R to include all of the elements in that index. The following code is the same as `dat$Lik1` because it includes all the rows and the specified column, `Lik1`.

```
dat[, "Lik1"]
```

```
## [1] 5 2 4 2 5 1 1 4 5 2
```



When we use indexing, we need to enclose the variable name in quotes. With data frames like this (in which we have multiple rows and columns), indexing is always in the order of row first and column second `[row, column]`.

Indexing might seem redundant because we can use the dollar sign, but it is more powerful. For instance, we might want to only examine the first three persons. We can use numbers in indexing (instead of column or row names, if we want). Here are the responses from the first three people in our data set on this question:

```
dat[1:3, "Lik1"]
```

```
## [1] 5 2 4
```

The following code is equivalent to the one above, but now we're using numeric indexing for the column. Note that `Lik1` is actually the second column in our data set, so we use 2 in the column index.

```
dat[1:3, 2]
```

```
## [1] 5 2 4
```

6.6 Creating objects with the combine function

The combine function, `c()`, is probably the most frequently used function in R. Here, we're combining several character strings and assigning them to an object, which we'll arbitrarily call `Likerts`.¹⁵ Because these are character strings, each element is in quotes.¹⁶ We can view the result by retyping the object we just created (and running it).

```
Likerts<- c("Lik1", "Lik2", "Lik3", "Lik4", "Lik5")  
Likerts
```

```
## [1] "Lik1" "Lik2" "Lik3" "Lik4" "Lik5"
```

¹⁵Just like SPSS, Excel, and other programs, R distinguishes character strings from numeric values.

¹⁶This is where the `dput(names(our data frame))` function comes in handy—we can manually copy part of our output from that and paste it into our code here without having to type the quotes and commas.



When we want to refer an object, we do *not* use quotes. Here, `Likerts` is an object. This is in contrast to our reference to the variable `Lik1` in the code above, `dat[, "Lik1"]`, which was enclosed in quotes because it was the name of that column.

Notice that this object, `Likerts`, includes the names of five Likert-type variables in our data set. We can look at this subset of data using the following code:

```
dat[, Likerts]
```

```
##      Lik1 Lik2 Lik3 Lik4 Lik5
## 1      5    5    4    3    2
## 2      2    2    3    1    2
## 3      4    4    3    3    2
## 4      2    2    2    1    2
## 5      5    5    3    5    4
## 6      1    1    2    2    3
## 7      1    2    3    1    1
## 8      4    1    3    4    5
## 9      5    3    4    4    3
## 10     2    2    3    3    4
```

We can also use `c()` with indexing. Let's examine only the first Likert-type question and the Teacher variable:

```
dat[, c("Lik1", "Teacher")] ]
```

```
##      Lik1 Teacher
## 1      5      No
## 2      2     Yes
## 3      4      No
## 4      2     Yes
## 5      5      No
## 6      1     Yes
## 7      1     Yes
## 8      4      No
## 9      5      No
## 10     2     Yes
```

It looks like all the teachers provided lower ratings on this question than their counterparts.

6.7 Putting the functions together

Here is some of the code we've addressed so far:

```
dat<- read.csv("SchoolSurvey.csv")
dat
head(dat)
tail(dat)
str(dat)
names(dat)
dput(names(dat))
Likerts<- c("Lik1", "Lik2", "Lik3", "Lik4", "Lik5")
Likerts
dat[, Likerts]
dat[, c("Lik1", "Teacher")] ]
```

Here is a typical set of functions for checking whether our data have been correctly imported:

```
dat<- read.csv("SchoolSurvey.csv")
dat
head(dat)
tail(dat)
str(dat)
```

7 Describing the data

7.1 The summary function

The `summary()` function is convenient. We can describe all of the variables in a data frame using the function on our data frame object, `dat`:

```
summary(dat)
```

##	PID	Lik1	Lik2	Lik3	Lik4
##	Min. : 1.00	Min. :1.00	Min. :1.00	Min. :2	Min. :1.00
##	1st Qu.: 3.25	1st Qu.:2.00	1st Qu.:2.00	1st Qu.:3	1st Qu.:1.25
##	Median : 5.50	Median :3.00	Median :2.00	Median :3	Median :3.00
##	Mean : 5.50	Mean :3.10	Mean :2.70	Mean :3	Mean :2.70
##	3rd Qu.: 7.75	3rd Qu.:4.75	3rd Qu.:3.75	3rd Qu.:3	3rd Qu.:3.75
##	Max. :10.00	Max. :5.00	Max. :5.00	Max. :4	Max. :5.00
##	Lik5	Teacher			
##	Min. :1.00	Length:10			
##	1st Qu.:2.00	Class :character			
##	Median :2.50	Mode :character			
##	Mean :2.80				
##	3rd Qu.:3.75				
##	Max. :5.00				

 Notice that we have also summarized the ID variable, which is not meaningful. We would obviously not include that in a report. This is simply a quick way to look at the data. If any of the min and max values were outside of our legitimate range on any of the variables, we would need to do some data cleaning. Also, we can see that *Teacher* is not a numeric variable. The results of the summary function with this variable do not return the mean, median and so forth because those would not make sense with non-numeric data. Instead, the frequency of observations in each category of this variable is reported.

We can apply the `summary()` function to a single variable:

```
summary(dat$Lik1)
```

##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
##	1.00	2.00	3.00	3.10	4.75	5.00

We can use many functions on subsets of data, using indexing. Which rows are included in the following summary function?

```
summary(dat[1:3,"Lik1"])
```

##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
##	2.000	3.000	4.000	3.667	4.500	5.000

Let's use `summary()` on the `Likerts` object we created above.¹⁷

```
summary(dat[,Likerts])
```

```
##           Lik1           Lik2           Lik3           Lik4           Lik5
## Min.      :1.00   Min.      :1.00   Min.      :2    Min.      :1.00   Min.      :1.00
## 1st Qu.:2.00   1st Qu.:2.00   1st Qu.:3    1st Qu.:1.25   1st Qu.:2.00
## Median :3.00   Median :2.00   Median :3    Median :3.00   Median :2.50
## Mean    :3.10   Mean    :2.70   Mean    :3    Mean    :2.70   Mean    :2.80
## 3rd Qu.:4.75   3rd Qu.:3.75   3rd Qu.:3    3rd Qu.:3.75   3rd Qu.:3.75
## Max.    :5.00   Max.    :5.00   Max.    :4    Max.    :5.00   Max.    :5.00
```

7.2 Specific functions for summarizing data

We can also perform functions like `min()`, `max()`, `median()` and assign them to objects that we can later use:



For each of the three lines, we performed the function on the variable and we assigned the output to an object, which we've arbitrarily named `minX`, `maxX`, and `medX`.

```
minX <- min(dat$Lik1)
maxX <- max(dat$Lik1)
medX <- median(dat$Lik1)
```

Let's view the objects in the console by typing them and running them:

```
minX
medX
maxX
```

Here's the output:

```
minX
## [1] 1
medX
## [1] 3
maxX
## [1] 5
```

Let's subtract `minX` from `maxX` to have a look at the range:

```
maxX - minX
## [1] 4
```

There are many functions we can use. For example, we can get the mean, variance, and standard deviation of a variable using these functions:

```
mean(dat$Lik1)
## [1] 3.1
var(dat$Lik1)
```

¹⁷Remember that we do not use quotes with objects (but that we did with names of variables).

```
## [1] 2.766667
```

```
sd(dat$Lik1)
```

```
## [1] 1.66333
```

A word of caution in working with Likert-type data such as these is worth our attention. We should probably not estimate the mean, variance, and standard deviation with individual Likert-type variables that have fewer than five categories because these types of functions assume the variables are continuous—that is, we are assuming that the conceptual distance between any two neighboring points on the numbered scale is the same as the distance between any other two consecutively numbered points.¹⁸

8 Creating new variables

8.1 Creating scores from items

If our five Likert-type questions are asking about the same concept, we can combine them into a single composite score and save that as a new variable, which we'll call **Energy** in our data frame. Then we can perform functions, such as `summary()` and `var()` on that variable. It's worth noting that after we have this composite score based on multiple Likert-type items, we have a stronger case for assuming that our variable is continuous enough for performing operations like `mean()`, `var()`, and so forth; however, some statisticians will still frown against this practice because the summation procedure still assumes equal intervals in the Likert-type item data. In practice, however, with composite variables, this usually does not result in meaningful misinterpretations [Carifio and Perla, 2008, Norman, 2010].

```
dat$Energy<- apply(dat[,Likerts], MARGIN = 1, mean)
```

```
# Let's view the data frame:
```

```
dat
```

##	PID	Lik1	Lik2	Lik3	Lik4	Lik5	Teacher	Energy
## 1	1	5	5	4	3	2	No	3.8
## 2	2	2	2	3	1	2	Yes	2.0
## 3	3	4	4	3	3	2	No	3.2
## 4	4	2	2	2	1	2	Yes	1.8
## 5	5	5	5	3	5	4	No	4.4
## 6	6	1	1	2	2	3	Yes	1.8
## 7	7	1	2	3	1	1	Yes	1.6
## 8	8	4	1	3	4	5	No	3.4
## 9	9	5	3	4	4	3	No	3.8
## 10	10	2	2	3	3	4	Yes	2.8

```
# Let's perform the summary function on the new variable.
```

```
summary(dat$Energy)
```

##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
##	1.60	1.85	3.00	2.86	3.70	4.40

Each person now has a score on our new variable, called **Energy**, which is the mean across the five Likert-type items. It looks like the average energy level was 2.86 on the 1 to 5 scale.

How did we accomplish this? Let's break the code down so it makes sense:

Earlier we created the object `Likerts` using the `c()` function and the names of our variables of interest—the five Likert-type questions. In other words, we did this:

¹⁸Mean and variance are more appropriate with continuous, interval-level, variables than with ordinal level variables. Next in this tutorial, we create composite scores, which we can probably more comfortably treat as continuous.

```
Likerts<- c("Lik1", "Lik2", "Lik3", "Lik4","Lik5")
```

Then, we subsetting our data, using indexing, with the column index consisting of our object, `Likerts`, which comprises the names of the five variables:

```
dat[,Likerts]
```

We could have achieved the same result with the `c()` function directly embedded in the index, as in `dat[,c("Lik1", "Lik2", "Lik3", "Lik4","Lik5")]` but it would have made our line of code longer and harder to read. Also, we can re-use the `Likerts` object when we perform other functions (such as `head(dat[,Likerts])`).

What is new here is the `apply()` function. This function has three **arguments**. The first one specifies the data (rows and columns) we want to apply a function to, the second argument specifies the margin we seek to apply the function to, and the third argument specifies the function we wish to have applied (`mean` in our case).



An argument is a statement within a function. Simple functions, like `summary()` and `median()` take a single argument. The order of arguments in a function is important so R can distinguish them; otherwise, we need to explicitly identify the argument. The function above, with its three arguments made explicit is `dat$Energy<- apply(X = dat[, Likerts], MARGIN = 1, FUN = mean)`. The argument identifiers are `X`, `MARGIN`, and `FUN`. In our example, I made “`MARGIN =`” explicit to bring our attention to it but we can omit the words “`MARGIN =`” and simply use the value (1 in our case) because it is the second argument in the function.

In our example, our data are `dat[,Likerts]`, as specified by our first argument.

The second argument tells R to apply the function to the first margin (`MARGIN = 1`), which refers to the rows. (The second margin would apply the mean to the columns, which is not what we seek to do here.) Just as in indexing, the first margin refers to rows and the second margin to columns. In other words, we wish to apply the mean function *along each row* (the first margin) and across the columns to get a mean score for each row.¹⁹ The result is a vector of values.

Look again at our code and notice that the arguments are separated by commas:

```
dat$Energy<- apply(dat[,Likerts], 1, mean)
```

8.2 Appending variables to our data frame

Finally, because the result is the same length as the number of rows in our data frame, we can append it as a new column using the `$` symbol and a new name, “`Energy`”, that we create on the fly: `dat$Energy`.²⁰

Let’s again examine our new data frame to see the new variable we created. Notice that it is the last column. Also, let’s make sure the calculation was correct: For Person 1, the mean across the responses was $\frac{5+5+4+3+2}{5} = 3.8$, which is the same value reported under the `Energy` column.

```
head(dat)
```

```
##   PID Lik1 Lik2 Lik3 Lik4 Lik5 Teacher Energy
## 1    1    5    5    4    3    2      No    3.8
```

¹⁹We are *applying* the `mean()` function across Person 1’s responses to the Likert-type questions, then applying `mean()` across Person 2’s responses, and so forth until the last person. If we wanted to know the mean response on each column, such as for each variable across all the people, we’d specify `MARGIN = 2`.

²⁰We can name our new variable anything we like, as long as it begins with a character. Numbers, periods, and underscores, `_`, are also acceptable in variable names, but spaces and special characters should be avoided, as should words that are used for functions, such as `c` or `mean`. Remember that R is case sensitive!

```
## 2  2  2  2  3  1  2  Yes  2.0
## 3  3  4  4  3  3  2   No  3.2
## 4  4  2  2  2  1  2  Yes  1.8
## 5  5  5  5  3  5  4   No  4.4
## 6  6  1  1  2  2  3  Yes  1.8
```

Try this `apply()` function out on the same data and create a new variable called **EnergySum** (or use whatever variable name you like, as long as it begins with a character). In other words, instead of the mean, calculate a total score based on the sum of responses across the Likert-type questions.²¹ You should see this as the result:

```
##   PID Lik1 Lik2 Lik3 Lik4 Lik5 Teacher Energy EnergySum
## 1   1   5   5   4   3   2     No    3.8         19
## 2   2   2   2   3   1   2     Yes    2.0         10
## 3   3   4   4   3   3   2     No    3.2         16
## 4   4   2   2   2   1   2     Yes    1.8          9
## 5   5   5   5   3   5   4     No    4.4         22
## 6   6   1   1   2   2   3     Yes    1.8          9
```

9 Creating a simple box plot

R provides many options for creating plots.²² Let's start simple and create a box plot. Because our **Teacher** variable is a factor, we can use a box plot to see whether the range of responses on our composite variable, **Energy**, depends on whether or not the school professional is a teacher.²³

```
par(bty="l")
boxplot(dat$Energy ~ dat$Teacher,
        ylim=c(1,5),
        ylab="Energy level",
        xlab="Is the respondent a teacher?")
```

We see that the teachers provided lower responses on the Likert-type questions than non-teachers. It seems (in our hypothetical data example) that teachers have lower energy levels than the other school staff on a Wednesday afternoon.

10 Getting help and troubleshooting

10.1 R's help files

To find out more about a function, use the `help()` function. A shortcut for this is `?`. For instance,

```
help("apply")
# Or, equivalently,
?apply
```

²¹With composite scores calculated from multiple items that all use the same scale, you might find the mean to be more informative than the sum because the composite score can be interpreted on the same scale. For instance, if each of our items' response scales was 1 = *Very low energy*, 2 = *Moderately low energy*, 3 = *Medium level of energy*, 4 = *Moderately high energy*, and 5 = *Very high energy*, we can interpret Person 1's score of 3.8 on **Energy** as being just below a moderately high energy level.

²²This example uses the base R graphics. We can get fancier plots using the `ggplot2` package.

²³The tilde, `~`, is used in R to indicate a formula. We will see this in regression and other statistical equations, where it serves the same role as the "=" operator (e.g., $Y \sim b_0 + b_1X$). Here, we're asking R to make a box plot with a response (or outcome) variable, `dat$Energy`, based on the independent (or explanatory) variable, `dat$Teacher`. To match the scale of the Likert-type items, we set the scale of the Y axis to go from 1 to 5 using the `ylim=` argument. We have also specified the X and Y axis labels with the `xlab=` and `ylab=` arguments. The line `par(bty="l")` is for aesthetics. `bty` stands for box type and the `l` is a lowercase L to tell R to print only the left and bottom lines of the box (like the shape of the letter L).

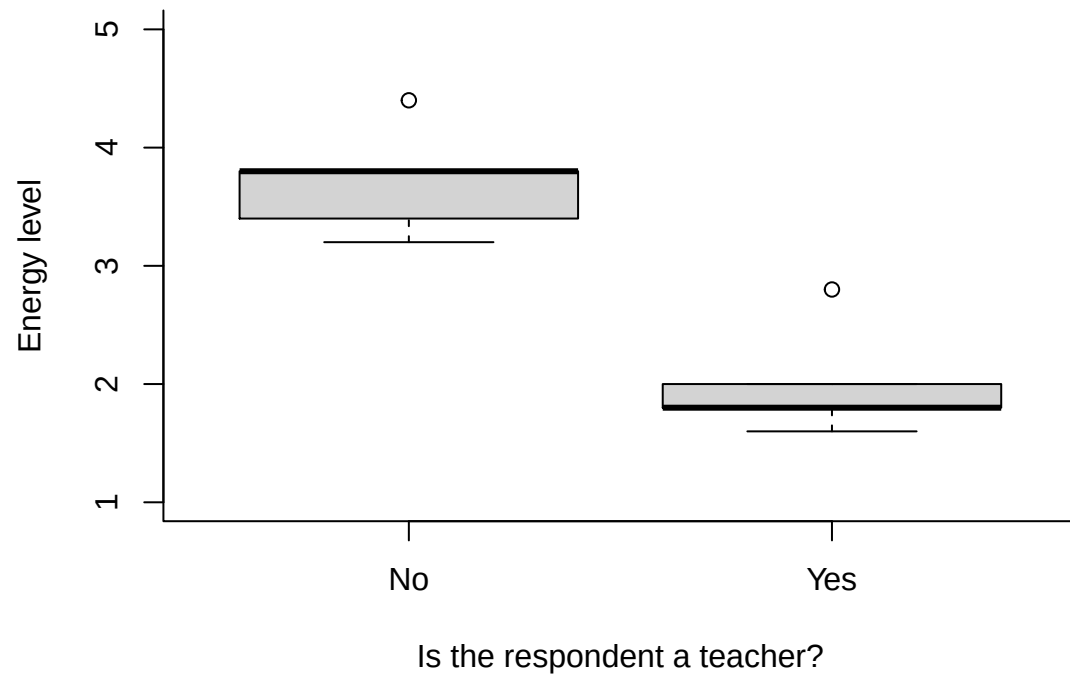


Figure 10: Boxplot of Teachers on X axis and Energy level on Y axis.



The # sign is used to comment out code in R, so the line that says “# Or, equivalently,” is not evaluated.

The help pane of RStudio should display information about the function we have placed in quotes or have placed after the question mark (if we’ve spelled it correctly). This is useful for seeing what arguments the function takes.²⁴. There is often a lot of useless technical information in R help files. However, if we scroll to the bottom of a help file, we can usually find examples.

If you want to see a list of the help files in a particular package, try `help(package = "package name")`. For example, if we wanted to see the help files for the `psych` package, we would run this code. Note that we would have to install the package and bring it into our library first (stuff for another tutorial).

```
help(package = "psych")
```

10.2 Online help

Finally, there is a lot of information online. We can search for something like **How to use the apply function in R** in a search engine and find results. If you conducted that search, you may have come across this site, which provides a nice visual of how the `apply()` function works when we use `MARGIN = 2`: <https://www.datacamp.com/community/tutorials/r-tutorial-apply-family>

There’s even a search engine specifically for R, RSeek at <https://rseek.org/>. Common sites that result from web searches are stackoverflow.com and stats.stackexchange.com. RStudio provides a useful introduction page, too, among other resources on their site.

10.3 Why won’t my code run?

One of the most frequent problems new users of R experience is when they have run only part of a section of code (such as only the `2 +` in the example below) and the console is waiting for them to complete the code. R expects another number. But, when you run another line of code that you are so sure is correct, such as `dat<- read.csv("SchoolSurvey.csv")`, you keep getting errors. Annoying!

You can tell if this is the problem if the console has a `+` at the start of the line. It is expecting something to be added to complete the code. In the example here, R could not understand `2 + dat<- read.csv("SchoolSurvey.csv")`, so it returned an error.

The solution is to place your cursor in the console and then press **ESC** on your keyboard. Now try your code.

Another strategy is to clear your environment (as described in Part 4) and re-run your code. If we have objects in our R session that have mistakes, they will remain there until we replace them with the correct objects. Clearing the environment ensures they’re gone. If you want even more peace of mind, clear your environment, save your R image, and reopen it to run your shiny new code. One troubleshooting strategy, after having cleared your environment, is to run portions of your code to isolate the error.

Finally, if we have error messages in our console, we can copy portions of that message and paste them within quotes into an online search, such as in <https://rseek.org/>. This sometimes reveals a solution to our specific problem.

10.4 Have patience and keep learning

There will be times in our learning of R that we experience frustration. Because it is open source, it is a bit of a wild west environment. Package developers differ in their coding and what they expect us to know about their coding. So, what works with one function in one package might not work with another. We also might discover new workflows as we familiarize ourselves with R. The code in this tutorial is all from R’s

²⁴However, a specific function for seeing what a function’s arguments are is the `args` function, for example, `args(apply)`

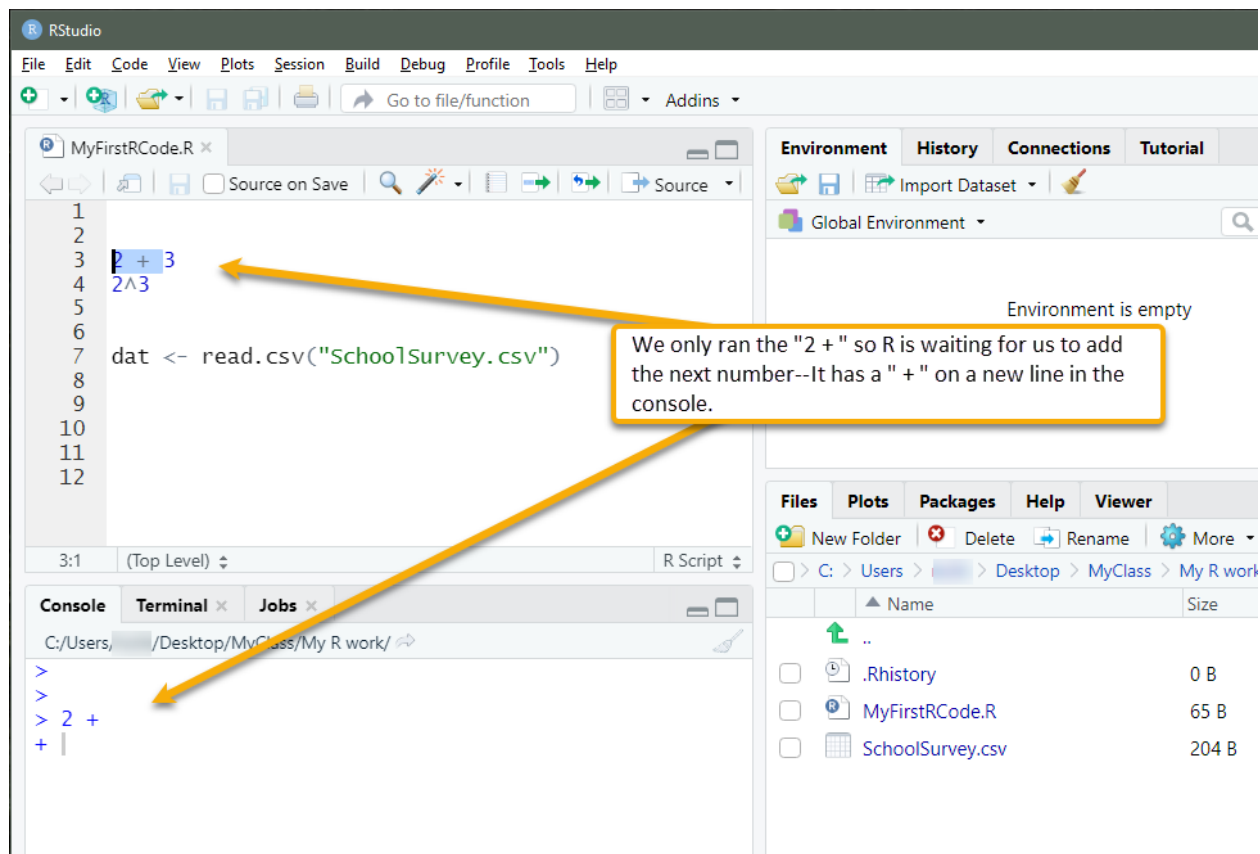


Figure 11: R is waiting for your to enter the next number

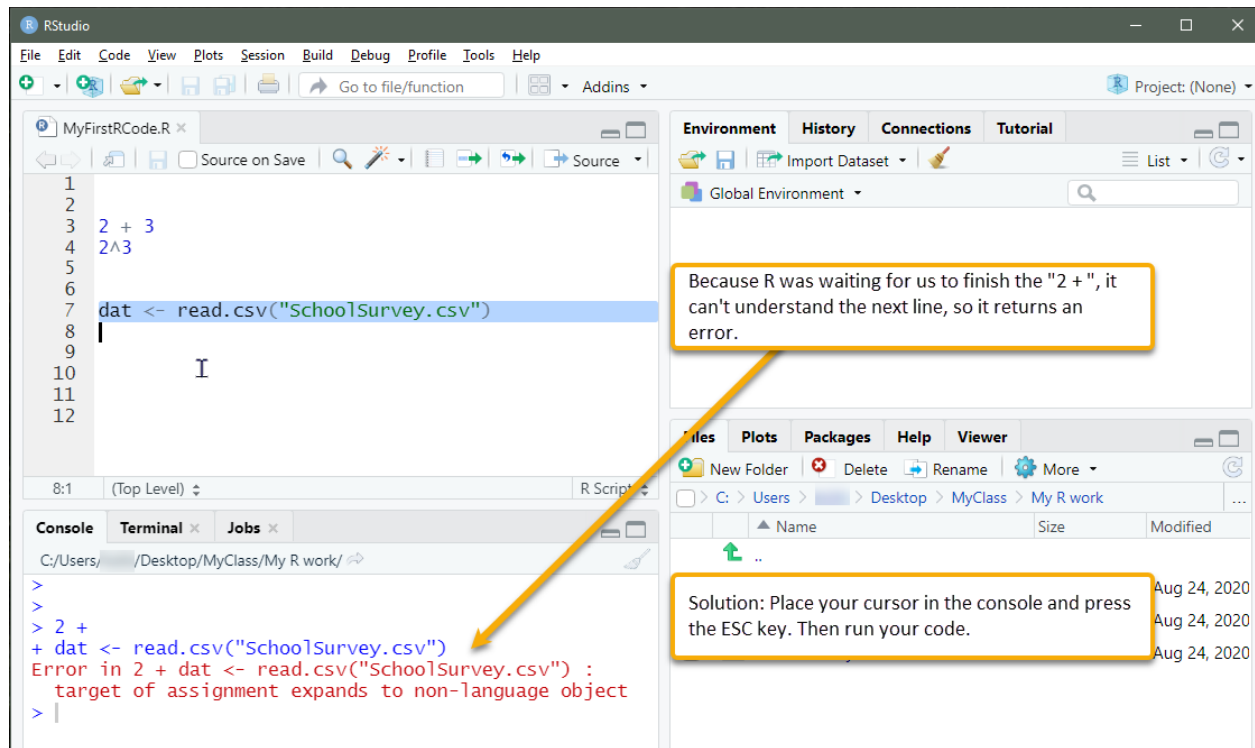


Figure 12: Why doesn't my code work?

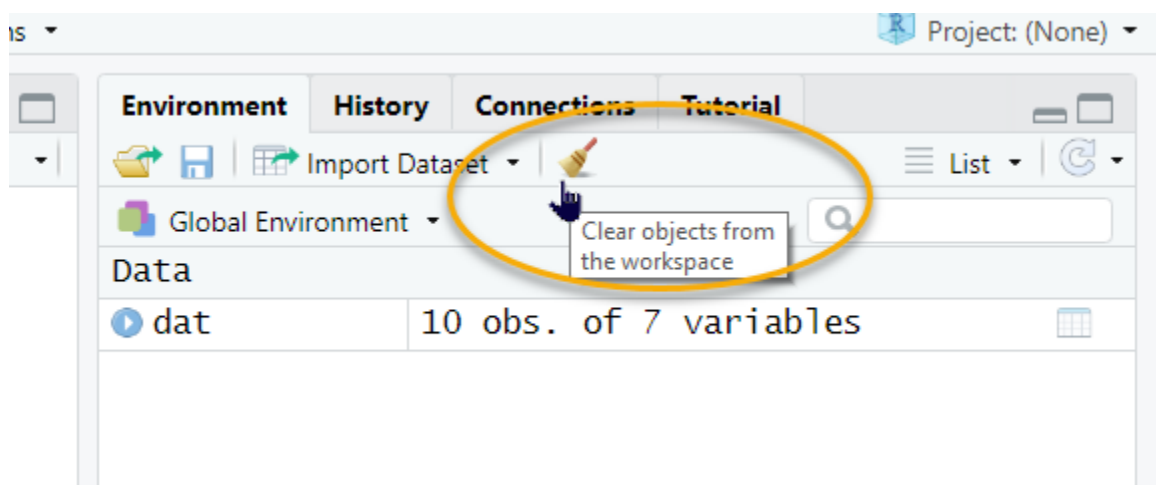


Figure 13: Clearing the environment removes previously made objects

base R code. In other words, we did not rely on any packages. Another approach to coding in R is from the Tidyverse collection of packages and developers, who strive to tidy up this wild west environment. Though Tidyverse is powerful, it still has limitations. It is wise to learn the basics of base R's language even if we wish to join the Tidyverse because many packages may be very useful yet not use Tidyverse conventions.

R is not only open source; it is very powerful. Although proprietary statistics programs are easier to use, we limit our capacity when we commit to their workflows. With R, collaboration does not require everyone pay for a license—there is no license. Published articles often include R code in their appendices for readers to use for their own work, which is powerful for continuing our data-analysis learning beyond our university education experience. Also, say you want to use what you learn in graduate school later on in your professional work. If your workplace does not pay for that proprietary license, you're at a loss. This is not a concern with R. Save your R files for adaptation for your future needs.

We can accomplish almost all of the same things in R as we can with proprietary programs. It also allows us to do things that otherwise would require multiple applications. We can manage all of our data and outputs from our analyses in a single project and save these in a reproducible format (i.e., our script). In contrast, with proprietary programs, we might have to juggle files between programs, which invites human error.²⁵ Once we have a comfortable R workflow, we can continue learning new things in R and contribute our findings to the world.

11 List of functions and other code in this tutorial

Here are the important functions and code we used in this tutorial:

- `read.csv()` for importing data from a CSV file
- `head()` and `tail()` for taking a peek at the first 6 and last 6 rows of data
- `str()` for examining the structure of the data
- `c()` for combining elements into a vector
- The `$` as in `dataset$variable` for selecting a specific variable in a data frame
- The use of brackets after a data frame, `dataset[ , ]`, for indexing a specific row or column or cell
- `summary()` for requesting a basic summary description of the variables
- `min()`, `mean()`, `median()`, `max()`, `var()`, and `sd()` for requesting specific statistics about a variable
- `apply()` for applying a function across a row or column of data
- `help()` or `?` for getting help on a function
- `boxplot()` for creating box-and-whisker plots with base-R graphics
- The ESC key when the console is expecting us to complete some code.

Updated on 2021-01-13.

References

- James Carifio and Rocco Perla. Resolving the 50-year debate around using and misusing likert scales. 42: 1150–1152, 2008. ISSN 03080110. doi: 10.1111/j.1365-2923.2008.03172.x.
- Greg Freedman Ellis and Ben Schneider. *srvyr: dplyr-Like Syntax for Summary Statistics of Survey Data*, 2020. URL <https://CRAN.R-project.org/package=srvyr>. R package version 1.0.0.

²⁵For instance, we might manage data in one program, such as SPSS, save that data into a format for another program, such as MPlus, import that file into MPlus, then export those data back into another program, such as SPSS for prettier graphics. In R, all of our steps are pretty much in the same place.

- Thomas Lumley. *survey: Analysis of Complex Survey Samples*, 2020. URL <http://r-survey.r-forge.r-project.org/survey/>. R package version 4.0.
- Geoff Norman. Likert scales, levels of measurement and the “laws” of statistics. 15:625–632, 2010. ISSN 1382-4996, 1573-1677. doi: 10.1007/s10459-010-9222-y. URL <http://link.springer.com/10.1007/s10459-010-9222-y>.
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2020. URL <https://www.R-project.org/>.
- William Revelle. *psych: Procedures for Psychological, Psychometric, and Personality Research*, 2020. URL <https://personality-project.org/r/psych/><https://personality-project.org/r/psych-manual.pdf>. R package version 2.0.12.
- RStudio Team. *RStudio: Integrated Development Environment for R*. RStudio, PBC., Boston, MA, 2020. URL <http://www.rstudio.com/>.